Relatório de Melhorias — Que Horas Passa (Linha C5)

Oct 14, 2026 · @Igor

Sumário executivo

O Que Horas Passa precisa de seis mudanças técnicas para que cada horário exibido seja rastreável, validado e corrigido pela operação real. Este relatório complementa o diagnóstico anterior e detalha **como** implementar as correções, sem reescrever o sistema.

As cinco prioridades, em ordem de custo/benefício:

1. **Banco como guardião da integridade:** constraints, `horario` tipado e PUT atômico, para nunca existir grade vazia ou corrompida.
2. **Cálculo +13/+32 min no servidor**, com `grade_hash` em toda resposta, eliminando valores concorrentes para a mesma viagem.
3. **Segurança básica do admin:** autenticação, RLS no Supabase, verificação de XSS e rate limit nas escritas.
4. **Testes com relógio falso**, incluindo a regressão do caso 05:50.
5. **Estimativa adaptativa** alimentada pelo botão "o ônibus passou agora", ligada somente após validação em modo sombra.

A grade oficial continua imutável. Toda correção de previsão é uma camada separada, marcada como estimativa e com faixa de incerteza.

Contexto e evidência

O caso que motiva tudo é a saída das 05:50: o sistema calcula 06:03 (Bela Vista) e 06:22 (Terminal), mas um usuário esperou por 06:30, 06:36 e 06:39, valores que não existem no banco, no fallback nem no código. O sistema não consegue explicar de onde veio a informação.

Cada melhoria abaixo ataca ao menos um ponto fraco do diagnóstico anterior:

Melhoria	Pontos fracos atacados
Constraints, tipo `TIME`/minutos, dia de serviço	R2, R3, R5
Cálculo +13/+32 no servidor + `grade_hash`	C1, C2, R4
PUT atômico e fallback gerado no build	R1, M2

Melhoria	Pontos fracos atacados
`/api/time` e fuso `America/Belem`	C5
Alerta com agendamento exato e Web Push	C4
Estimativa adaptativa	C3, M1
Auditoria por trigger e migrations	M3

Integridade de dados no banco

As regras de validade devem morar no Postgres, porque a API pode falhar ou ser contornada e o banco não.

- **Constraints:** `CHECK` nos dias, `UNIQUE` contra viagens duplicadas e `NOT NULL` nos campos críticos.
- **Horário tipado:** `TIME` ou minutos desde 00:00 (inteiro), sem truncar string `HH:mm` e sem ambiguidade de segundos ou fuso (R3).
- **Dia de serviço, como no GTFS:** uma viagem que sai às 23:50 e termina às 00:20 pertence ao dia anterior, o que resolve a virada de meia-noite (R5) na modelagem.
- **+13/+32 min calculados no servidor,** em view ou coluna gerada. O cliente apenas exibe, o que elimina as divergências C2 e R4.
- **Migrations versionadas** em `supabase/migrations`, para o esquema ser reproduzível e auditável.

```
ALTER TABLE horarios
  ADD CONSTRAINT dias_validos
    CHECK (dias <@ ARRAY['seg','ter','qua','qui','sex','sab','dom']),
  ADD CONSTRAINT uq_viagem UNIQUE (sentido, horario, dias);
```

Critério de aceite: inserir um horário duplicado ou um dia inválido diretamente no banco, fora da API, deve ser rejeitado.

API

A API passa a ser a garantia de consistência e de rastreabilidade entre banco e cliente.

Mudança	Problema que resolve	Detalhe
PUT atômico via função RPC no Postgres	Grade vazia se o PUT falhar no meio (R1)	Valida e troca tudo dentro de uma transação

Mudança	Problema que resolve	Detalhe
`ETag` + `Cache-Control: s-maxage` com `stale-while-revalidate`	Lentidão em rede ruim	Evita servir dado velho indefinidamente
`grade_hash` em toda resposta	Falta de rastreabilidade (C1)	Qualquer relato de erro é reproduzível contra a versão exata
Endpoint `/api/time`	Relógio errado do aparelho (C5)	O cliente calcula o deslocamento uma vez e o aplica
Timeout com `AbortController` e retry com backoff exponencial e jitter	API lenta parece "sem horários" (M2)	Limite de tentativas e mensagem de erro clara
Fuso `America/Belem` via `Intl.DateTimeFormat`	`new Date()` depende do dispositivo	Pará usa UTC-3, sem horário de verão

Critério de aceite: simular falha no meio do PUT e confirmar que a grade anterior permanece intacta e consultável.

Fallback e motor de alerta

O fallback deixa de ser uma cópia manual que diverge do banco, e o alerta deixa de depender de uma aba aberta.

Fallback sem divergência

- **Eliminar `horariosFallback` hardcoded.** Gerar um `fallback.json` no build a partir do banco, ou guardar o último payload bom em `localStorage`/Cache Storage (*last-known-good*), com `grade_hash` e data de captura.
- **Ativar também em lista vazia,** com faixa de aviso visível (R1).
- **Teste no CI** que compara `fallback.json` com a produção e falha o pipeline se houver drift.

Motor de alerta

- **Agendamento exato** em vez de polling de 30 s: calcular o timestamp-alvo, usar `setTimeout(alvo - agora)` e recalcular em `visibilitychange` e `pageshow`, para cobrir aba suspensa.

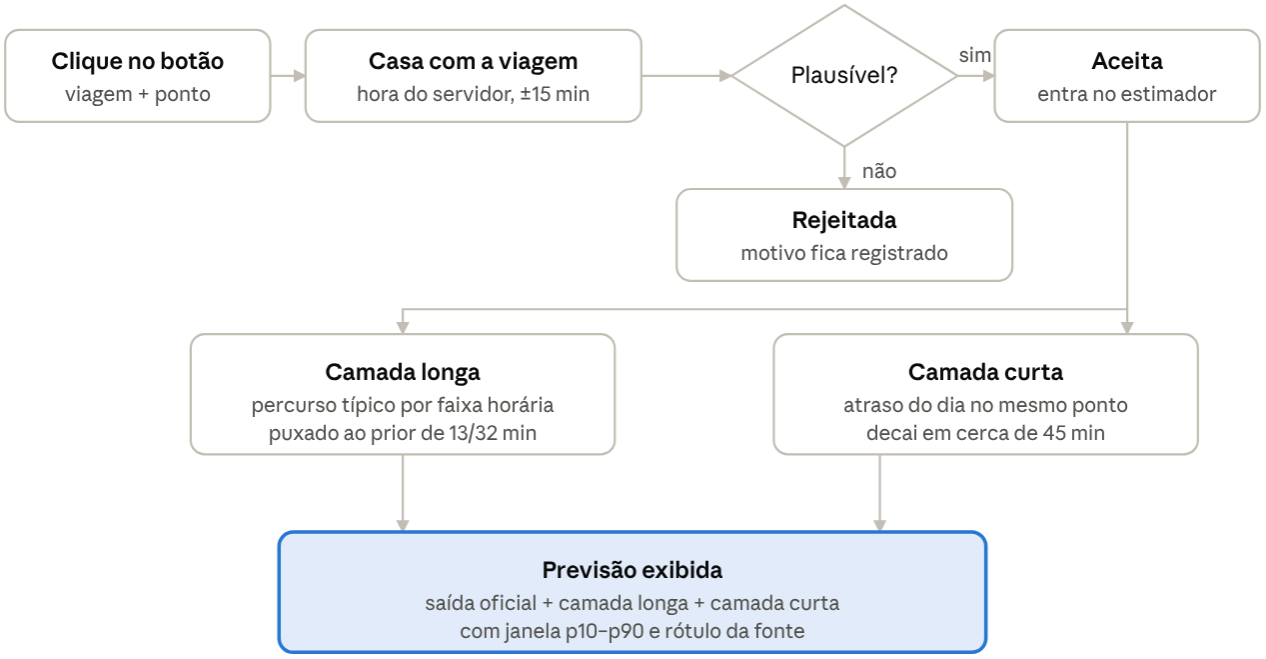
- **Web Push (VAPID) com Service Worker,** disparado por um agendador (`pg_cron` ou Edge Function). É a única solução robusta para C4, pois funciona com a aba fechada.
- **Idempotência** com chave `(dispositivo, viagem, data)`, para não notificar duas vezes nem perder um alerta em reconexão.

Critério de aceite: com a aba em segundo plano por 20 minutos, o alerta ainda chega no minuto configurado, uma única vez.

Estimativa adaptativa com o botão "o ônibus passou agora"

Cada clique vira uma observação de tempo de percurso real, e um estimador robusto corrige os +13/+32 min fixos sem alterar a grade oficial.

Cada clique corrige a previsão sem tocar na grade oficial



estimativa adaptativa · 2 camadas, 1 filtro

O clique envia só a viagem e o ponto; a hora vem do servidor. Observações plausíveis alimentam uma camada estrutural (percurso típico) e uma de atraso do dia.

Modelo de dados

```
CREATE TABLE observacoes_passagem (
  id              bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  viagem_id       bigint NOT NULL REFERENCES horarios(id),
  ponto           text   NOT NULL CHECK (ponto IN ('bela_vista','terminal')),
  data_servico    date   NOT NULL,
  observado_em    timestamptz NOT NULL DEFAULT now(),
  percurso_s      int    NOT NULL,
  residuo_s       int,
  grade_hash      text   NOT NULL,
  device_hash     text   NOT NULL,
  status          text   NOT NULL DEFAULT 'pendente'
                  CHECK (status IN ('pendente','aceita','rejeitada')),
  motivo_rejeicao text,
  UNIQUE (device_hash, viagem_id, ponto, data_servico)
);
```

A tabela `estimativa_percurso` guarda, por chave (linha, ponto, tipo de dia, faixa horária), a mediana, p10, p90, `n` e a estimativa final já com *shrinkage* e limite. O `UNIQUE` garante um voto por dispositivo, viagem, ponto e dia direto no banco.

Estimador

A camada longa usa hierarquia de buckets com recuo: viagem, faixa de 30 min, tipo de dia + ponto e, por fim, o prior fixo. Cada nível é puxado em direção ao nível pai:

```
est_{nivel} = \frac{W \cdot mediana_{pond} + k \cdot est_{pai}}{W + k}
```

A camada curta reflete o atraso ocorrido agora e decai com o tempo:

```
ajuste(t) = residuo \cdot e^{-(t - t_{obs}) / \tau}
```

Parâmetro	Valor sugerido	Função
`k`	≈ 5	Teimosia do prior: 1 ou 2 cliques quase não movem a estimativa
Peso por idade	meia-vida de 14 dias	Observações antigas pesam menos
Janela	30 dias	Período considerado na camada longa
`τ`	≈ 45 min	Decaimento do ajuste do dia; vale por no máximo 60 min

Parâmetro	Valor sugerido	Função
Limite da camada longa	±10 min do prior	Desvio maior vira alerta ao admin, não ajuste automático
Limite da camada curta	±8 min	Evita que um único atraso distorça as viagens seguintes
`N_min`	5	Abaixo disso a fonte é rotulada `prior`; acima, `adaptativa`

Ingestão e resistência a abuso

- **Hora do servidor** em `observado_em`; o relógio do aparelho é ignorado (C5).
- **Casamento ambíguo** (duas viagens a menos de 3 min de diferença): o cliente pergunta qual foi, ou o servidor rejeita.
- **Filtros de plausibilidade:** percurso fora de `[prior − 10 min, prior + 20 min]`, dia sem viagem programada ou excesso de observações por dispositivo.
- **Outlier estatístico:** rejeitar se `|percurso − mediana| > 3 × MAD` quando `n ≥ 8`. Mediana e MAD impedem que um cliente malicioso desloque o resultado sozinho.
- **Quórum de dispositivos distintos** antes de sair do prior, com peso menor para dispositivos novos.
- **Auditoria:** `status` e `motivo_rejeicao` ficam registrados, e o admin pode invalidar uma observação sem apagá-la.

Validação antes de ligar

1. **Modo sombra:** calcular e gravar a estimativa adaptativa, mas continuar exibindo o prior fixo.
2. **Backtesting** com retenção temporal (treina até D−7, avalia de D−7 a D), comparando o erro absoluto médio (MAE) do prior e do adaptativo.
3. **Feature flag** por linha, com rollback imediato para o prior.
4. **Métricas contínuas:** MAE por faixa, cobertura de buckets com `n ≥ N_min`, taxa de rejeição e taxa de ambiguidade no casamento.

Critério de aceite: ligar a estimativa adaptativa somente se o MAE dela for menor que o do prior de forma consistente no backtesting.

Segurança, testes e observabilidade

Segurança

- **Auth real no admin:** Supabase Auth + RLS, com a chave `service_role` apenas no servidor, nunca no cliente. POST e PUT são superfície crítica, pois quem altera a grade altera o que o usuário espera na parada.
- **Verificar XSS:** se `destino` ou outros campos vão para a tela via `innerHTML`, texto cadastrado no admin vira vetor. Usar `textContent` ou sanitização.
- **Rate limiting** nos endpoints de escrita e CORS restrito à origem do app.
- **Auditoria por trigger** em `horarios_audit` (operação, linha antiga e nova, usuário, timestamp), sem depender de a aplicação lembrar de registrar.

Testes e CI

- **Relógio falso** (`vi.useFakeTimers` no Vitest/Jest) para testar alerta, virada de dia, +13/+32 e fuso de forma determinística.
- **Regressão do caso real:** para a saída 05:50, o sistema deve gerar 06:03 e 06:22 e não tratar 06:30, 06:36 e 06:39 como pertencentes à grade.
- **Testes de propriedade** (`fast-check`) para ordenação e comparação de horários na virada de dia.
- **Contrato da API** com schema compartilhado (Zod/JSON Schema) entre admin, API e cliente, e e2e com Playwright simulando relógio e fuso.

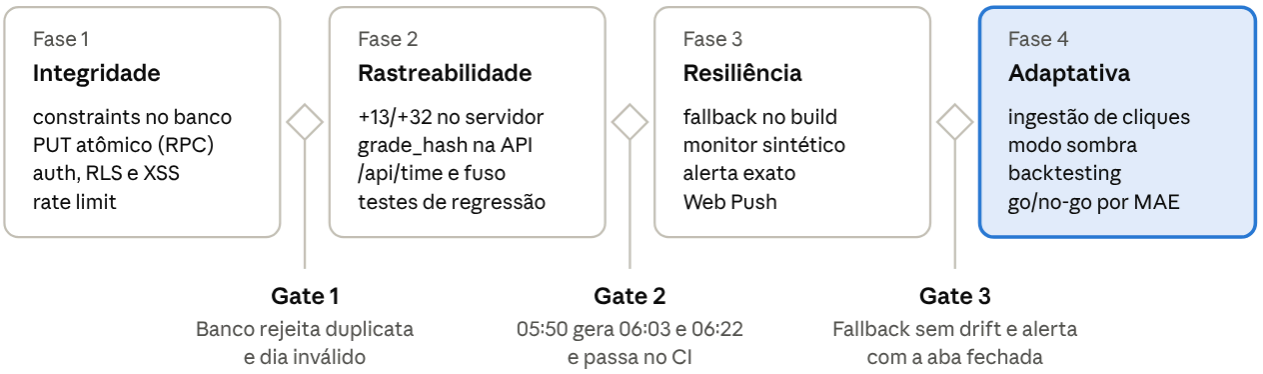
Observabilidade

- **Logs estruturados** com `request_id` e `grade_hash` em cada resposta e em cada alerta.
- **Captura de erros no front** (Sentry ou equivalente), incluindo falhas de fetch e de parse de horário.
- **Monitor sintético** a cada 5 minutos: chama a API, valida o schema, confere que a lista não está vazia e compara o hash esperado, avisando o admin em caso de falha.
- **Métricas:** taxa de fallback, latência p95 da API, erros de validação e divergência de relógio cliente/servidor.

Plano de implementação

A implementação segue quatro fases, e cada fase só começa depois de o critério de aceite da anterior passar.

Três critérios de aceite separam as quatro fases



Destaque: a fase 4 só é ligada se o MAE adaptativo for menor que o do prior.

roadmap · 4 fases, 3 gates

A ordem põe primeiro o que evita o pior cenário (grade vazia ou corrompida) e deixa por último o que depende de dados acumulados (estimativa adaptativa).

Prioridade	Entrega	Pontos fracos atacados	Esforço relativo
1	Constraints no banco e PUT atômico	R1, R2, R3	Baixo
2	+13/+32 no servidor e `grade_hash`	C1, C2, C3, R4	Médio
3	Auth, RLS, XSS e rate limit	Superfície de escrita do admin	Médio
4	Testes com relógio falso e caso 05:50	C5, R5	Baixo
5	Fallback no build e monitor sintético	R1, M2	Médio
6	Web Push	C4	Alto
7	Estimativa adaptativa	C3, M1	Alto

Riscos e mitigações

- **Poucos cliques para calibrar:** a estimativa permanece no prior (`fonte: prior`) até `N_min` , e o usuário vê o rótulo.

- **Observações falsas ou duplicadas:** unicidade no banco, mediana com MAD e quórum de dispositivos distintos.
- **Web Push depende de permissão do usuário:** manter o agendamento exato no cliente como alternativa.
- **Grade errada vista como erro de percurso:** o limite de ±10 min gera alerta ao admin em vez de ajuste automático.